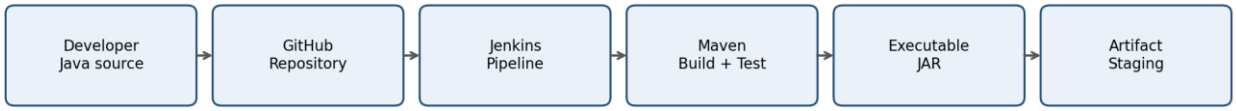
Student Academic Management Platform using DevOps Automation

Version Control, Automated Build, Unit Testing, Packaging & Artifact Staging



Document Attribute	Specification
Project	Java Maven CI/CD Pipeline
Application	my-java-app
Technology Stack	Java 21, Maven, Jenkins, GitHub, Linux/EC2
Build Automation	Jenkins Declarative Pipeline
Artifact	my-java-app-1.0-SNAPSHOT.jar
Project Repository	https://github.com/anushkarajdev2410-netizen/java_mvn
Document Purpose	Academic project explanation, architecture, implementation, validation and evidence
Document Version	v1.0 — Prepared from the project details shared in the conversation

1. Executive Summary & Engineering Context

This project demonstrates a practical Continuous Integration and delivery-oriented workflow for a Java application. The source code is maintained in GitHub, while Jenkins automates checkout, Maven build execution, test reporting, JAR packaging and artifact staging. The application uses Java 21 and Maven, and the pipeline is hosted on a Linux EC2 environment running Jenkins.

The project is intended to reduce repetitive manual work and provide a repeatable path from source code to a packaged application artifact. The successful Jenkins build shown during project validation is build #15. Jenkins displayed the JAR artifact and the Latest Test Result page reported no failures. The generated JAR was also executed locally and printed: “Hello, Jenkins CI/CD Pipeline!”

Project outcome: A working Jenkins pipeline that produces an executable Java JAR. The pipeline's current deployment step should be described as artifact staging unless a running application server or production environment is separately configured.

1.1 Project Objectives

- Store application source code and pipeline configuration in GitHub.
- Use Jenkins to retrieve the repository and run repeatable build steps.
- Use Maven to compile Java source code and execute unit tests.
- Package the application as a versioned JAR file.
- Publish the JAR as a Jenkins build artifact and make it available for download.
- Record build status and test results to support troubleshooting and academic evidence.

1.2 Scope and System Boundaries

Included in this project	Boundary / clarification
GitHub repository and Jenkinsfile	Repository checkout is automated by the pipeline configuration.
Java 21 and Maven build	Build commands execute within the project directory containing pom.xml.
Unit test reporting	Jenkins can display test results when Maven generates Surefire XML reports.
JAR packaging and archiving	The resulting JAR is available from the Jenkins job page.
Artifact staging	Copying a JAR into a deployment folder is staging, not proof of live production deployment.

2. Business Need for DevOps & CI/CD

2.1 Traditional Delivery Challenges

When developers build and package software manually, each release depends on a sequence of individual actions. A missed command, an incorrect working directory, a different Java version, or an overlooked test can produce an artifact that behaves differently from the developer's local result. Manual handoffs also make it harder to identify which source revision produced a particular package.

- Environment inconsistency: Java or Maven versions can differ between a workstation and a build server.
- Build repetition: developers repeatedly run the same commands by hand.
- Late defect discovery: errors may be found only after packaging or deployment.
- Weak traceability: the relationship between a commit, build result and JAR may be unclear.
- Artifact handling errors: files can be misplaced, overwritten or downloaded from an older build.

2.2 How CI/CD Addresses These Challenges

Dimension	Manual workflow	Project CI/CD workflow
Source retrieval	Developer manually downloads or copies source	Jenkins checks out source from GitHub
Build	Commands run by hand	Maven commands run in Jenkins stages
Testing	May be skipped or run inconsistently	Pipeline can execute tests and publish reports
Packaging	Manual creation and copying	Maven creates a versioned JAR; Jenkins archives it
Traceability	Depends on personal notes	Build number, console output, test report and artifact are visible in Jenkins
Repeatability	Sensitive to operator steps	Pipeline-as-code describes the sequence of actions

3. Project Architecture & Workflow

The architecture separates source control, automation, build tooling and artifact delivery. GitHub is the source repository; Jenkins coordinates the workflow; Maven resolves project configuration and runs Java build goals; the JAR is the packaged output.

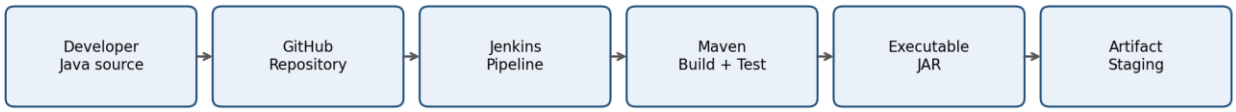


Figure 1. High-level architecture of the Java Maven CI/CD project.

3.1 Main Components

Component	Role in the project
Git / GitHub	Stores source files, pom.xml and Jenkinsfile; provides version history.
Jenkins	Runs the pipeline, displays stage status, records console output and publishes build artifacts.
Java 21 (JDK)	Provides the compiler and runtime used by the application.
Apache Maven	Reads pom.xml, manages build lifecycle goals and creates the JAR.
JUnit 5	Provides the test framework dependency for automated unit tests.
Jenkins artifact archive	Makes the built JAR downloadable from the Jenkins job/build page.
Linux EC2 instance	Hosts the Jenkins service and executes pipeline steps in the configured environment.

3.2 Source Layout

The project has been referenced in the following repository path layout:

- Desktop/my-java-app/Jenkinsfile — Jenkins pipeline definition.
- Desktop/my-java-app/pom.xml — Maven project configuration and plugins.
- Desktop/my-java-app/src/main/java/com/example/App.java — Java application source.

- Desktop/my-java-app/src/test/java/ — expected location for Java unit tests, if present.

The pipeline must run Maven from the directory that contains pom.xml. If Jenkins checks out the repository with the project nested under Desktop/my-java-app, Maven steps should use that directory consistently.

4. Git and GitHub Version Control

4.1 Purpose of Version Control

Git records the history of project files and lets developers inspect or restore earlier versions. GitHub hosts the remote repository and makes the source available to Jenkins. Keeping the Jenkinsfile and pom.xml under version control helps make the build process reviewable and reproducible.

4.2 Typical Development Workflow

1. Edit Java source, test code, pom.xml or Jenkinsfile in the project directory.
2. Run relevant checks locally when possible.
3. Use git status and git diff to review changes.
4. Commit the changes with a meaningful message.
5. Push the commit to GitHub.
6. Run Jenkins manually or configure a repository trigger to build the updated revision.

4.3 Git vs. GitHub

Criterion	Git	GitHub
Type	Distributed version control tool	Hosted repository and collaboration platform
Where it runs	Local computer or server	Cloud-hosted web platform
Main function	Tracks changes, commits, branches and merges	Stores remote repositories and supports collaboration
Project role	Records source history	Provides the source Jenkins checks out

5. Jenkins Continuous Integration Pipeline

5.1 Jenkins Role

Jenkins is the automation server used to coordinate the project's build workflow. A pipeline written in a Jenkinsfile defines the stages and commands. The job page provides a visible status, build history, console output, test result trend and downloadable artifacts.

5.2 Pipeline Stages

Stage	Purpose	Expected evidence
1. Checkout Code	Retrieve the configured source revision from GitHub.	Checkout messages and repository revision in Console Output.
2. Build	Run Maven compilation from the project folder.	Successful Maven build output and compiled classes.
3. Unit Test	Run `mvn test` and publish Surefire reports.	Executed test count and test result report.
4. Package	Run Maven packaging and create the JAR.	JAR in the target directory and Jenkins archived artifact.
5. Deploy / Stage	Copy or place the artifact in the configured staging folder.	Successful copy command and artifact present in the target folder.

5.3 Important Implementation Checks

- Run Maven commands inside the directory containing pom.xml.
- Use `mvn test` in the Unit Test stage; `mvn clean compile` compiles code but does not run unit tests.
- Publish test reports from the actual project path, for example `Desktop/my-java-app/target/surefire-reports/*.xml`, when that matches the workspace layout.
- Avoid treating an empty test report as proof that tests passed; confirm that tests were discovered and executed.
- Ensure Package and Deploy stages use the same project and artifact paths as the Build stage.
- Archive the generated JAR so the artifact is attached to the Jenkins build.

Pipeline status: The Jenkins job screenshot shared for this project showed build #15 as the latest successful build, with the artifact `my-java-app-1.0-SNAPSHOT.jar` and a Latest Test Result link reporting no failures. Confirm the detailed test count and stage view when preparing final evidence.

6. Maven Build Configuration & Java Packaging

6.1 Maven Project Model

Maven uses pom.xml as the project object model. It identifies the artifact, declares dependencies and plugins, and configures the build lifecycle. The project coordinates shared in this assignment are groupId `com.example`, artifactId `my-java-app`, and version `1.0-SNAPSHOT`.

Setting	Project value / purpose
groupId	com.example — identifies the project group.
artifactId	my-java-app — identifies the application artifact.
version	1.0-SNAPSHOT — development version used in the JAR filename.
Java release	21 — configures the Java language/API level.
JUnit dependency	JUnit Jupiter 5 — test framework dependency.
Maven Compiler Plugin	Compiles the application's Java source.
Maven Surefire Plugin	Discovers and executes unit tests during `mvn test`.
Maven JAR Plugin	Packages compiled classes and can set the Main-Class manifest entry.

6.2 Build Lifecycle Commands

Command	What it does
`mvn clean`	Removes previous build output from target/.
`mvn compile`	Compiles main application source code.
`mvn test`	Compiles and runs configured unit tests.
`mvn package`	Runs earlier lifecycle phases and creates the JAR package.
`mvn package -DskipTests`	Packages the application while skipping test execution; use only when tests have already run in a separate stage.

6.3 Executable JAR and Main-Class

A JAR file is a ZIP-format package containing Java classes and metadata. For `java -jar` to launch an application, the manifest normally needs a Main-Class entry that identifies the class containing `public static void main(String[] args)`. The project's main class is `com.example.App`.

The application's observed output after running the packaged JAR was:

```
Hello, Jenkins CI/CD Pipeline!
```

This local execution is evidence that the downloaded JAR is executable in the tested Java environment. It does not by itself demonstrate that the application is running as a continuously hosted web service.

7. Testing, Quality Gates & Reporting

7.1 Why Automated Tests Matter

Automated tests provide repeatable checks of application behavior. In a Maven project using JUnit Jupiter, test classes are typically placed in src/test/java and test methods express expected behavior with assertions. Jenkins can consume XML test reports generated by Maven Surefire and present pass/fail trends.

7.2 Test Verification Checklist

- Confirm test source files exist under src/test/java.
- Run `mvn test` from the project root and check Maven's test summary.
- Confirm that `target/surefire-reports/` contains XML report files.
- Open Jenkins' Latest Test Result page and verify that tests were actually executed.
- Investigate failures using the report details and Console Output before treating the pipeline as complete.

7.3 Quality Gate Interpretation

Observed result	Interpretation
Build stage green	The stage's configured commands completed successfully.
Test report says no failures	No failures were recorded in the published report; verify test count and report presence.
JAR is archived	Jenkins retained the artifact and made it downloadable.
Deploy stage green	The configured deployment/staging commands completed; inspect the commands to establish what was actually deployed.

8. Artifact Delivery and Deployment Boundary

8.1 Artifact Archiving

Artifact archiving makes a build output available from Jenkins and associates it with a specific build number. In this project, the expected artifact name is `my-java-app-1.0-SNAPSHOT.jar`. This association helps the user identify which successful build produced the package.

8.2 Artifact Staging vs. Live Deployment

Activity	Meaning	What would prove it
Build and package	Maven produces a JAR file.	JAR exists under target/.
Archive artifact	Jenkins retains the JAR for download.	Artifact link on the Jenkins job/build page.
Copy to staging folder	JAR is copied to a configured directory.	Successful copy output and file listing on the host.
Live deployment	Application is installed or launched in a target runtime.	Service/process/container status, health check or reachable application endpoint.

Accuracy note: If the current Deploy stage only copies the JAR to a `deployment` folder, describe it as artifact staging. Claim live deployment only after verifying a running process/service or an accessible application endpoint.

8.3 Suggested Next-Level Improvements

- Trigger builds automatically from GitHub pushes using a webhook or multibranch pipeline.
- Add a test gate that fails the pipeline when tests fail or no tests are discovered.
- Use versioned artifacts or commit identifiers for release traceability.
- Move credentials into Jenkins Credentials rather than hard-coding secrets.
- For a real deployment, define the target runtime and add health checks, logs and a rollback plan.

9. Security, Reliability & Operational Considerations

- Credentials: never store AWS keys, GitHub tokens or passwords directly in Jenkinsfile or source code; use Jenkins Credentials.
- Network access: restrict the Jenkins web interface to trusted IP addresses or place it behind a secure access layer; use HTTPS for long-term use.
- Least privilege: grant the Jenkins host only the permissions needed to build and stage artifacts.
- Dependency hygiene: keep Maven dependencies and plugins updated and review vulnerabilities where practical.
- Build reproducibility: record Java and Maven versions and keep build configuration under version control.
- Retention: configure build and artifact retention so the Jenkins host does not fill its disk.
- Recovery: retain source in GitHub and document how to recreate the Jenkins job and its tool configuration.

10. Validation Results & Evidence

The following results are grounded in the project details and screenshots shared during troubleshooting. Recheck the current Jenkins instance before submission in case later builds have changed the latest build number.

Validation item	Current evidence / status
Jenkins job	Java-Maven-CI-CD
Latest successful build shown	Build #15
Build status	Successful / green status
Artifact displayed	my-java-app-1.0-SNAPSHOT.jar
Test result summary	Jenkins displayed “Latest Test Result (no failures)”; confirm the test count/report contents.
Executable JAR test	Printed `Hello, Jenkins CI/CD Pipeline!` when run locally.
Repository	https://github.com/anushkarajdev2410-netizen/java_mvn
Deployment status	Artifact staging should not be described as live deployment unless a runtime endpoint or running service is verified.

10.1 Screenshots Recommended for Submission

- Jenkins job overview showing the latest successful build number.
- Pipeline stage view showing the status of each stage.
- Console Output for the successful build.
- Latest Test Result page, including the number of tests and failures.
- Archived JAR artifact shown on Jenkins.
- Terminal output from executing the JAR.
- GitHub repository view showing Jenkinsfile, pom.xml and App.java.

11. Conclusion

The Java Maven CI/CD project demonstrates how source control and build automation can be combined to produce a repeatable software delivery workflow. GitHub maintains the code and pipeline definition, Jenkins orchestrates the workflow, Maven compiles and packages the application, and test reports and archived artifacts make results easier to inspect. The successful build and executable JAR provide evidence that the core build-and-package path is working. The next priorities are to verify that unit tests are genuinely executed and reported, and to clearly distinguish artifact staging from a fully operational deployment.

12. References & Project Resources

- Project repository: https://github.com/anushkarajdev2410-netizen/java_mvn
- Jenkins User Documentation: https://www.jenkins.io/doc/
- Jenkins Pipeline Syntax: https://www.jenkins.io/doc/book/pipeline/syntax/
- Apache Maven Getting Started Guide: https://maven.apache.org/guides/getting-started/
- Apache Maven Surefire Plugin: https://maven.apache.org/surefire/maven-surefire-plugin/
- Java Documentation: https://docs.oracle.com/en/java/
- JUnit 5 User Guide: https://junit.org/junit5/docs/current/user-guide/